Probabilistic View

- We consider probabilities
- For a dataset $D = \{(x_i, y_i)\}$ the likelihood (Probability that the data emerged via a certain model) is defined by

$$p(\{(x_i, y_i)\}|\theta).$$

- We want to maximize the posterior probability (Probability of a model given the data)

$$p(\theta|\{(x_i, y_i)\})$$

Probabilistische Sicht

- With Bayes theorem we get

$$p(\theta|\{(x_i, y_i)\}) \propto \underbrace{p(\{(x_i, y_i)\}|\theta)}_{\text{Likelihood}} \cdot \underbrace{p(\theta)}_{\text{Model Prior}}$$

- We did not consider output prior in this model.

- Instead of maximizing this probability we can equivalentlyx maximize the logarithm of it or minimize the negative logarithm

$$-\log p(\theta|\{(x_i, y_i)\}) = -\log p(\{(x_i, y_i)\}|\theta) - \log p(\theta)$$

Probabilistische Sicht

» The standard choice

$$p(\{(x_i, y_i)\}|\theta) = \prod_{i=1}^{N} p((x_i, y_i)|\theta) = \prod_{i=1}^{N} \exp(-||f_\theta(x_i) - y_i||\text{^}2)$$

gives

$$-\log p(\{(x_i, y_i)\}|\theta) = \sum_{i=1}^{N} ||f_\theta(x_i) - y_i||^2$$

Where I omitted the variances and equalities are only correct up to a multiplicative constant factor.

Probabilistic View

- Standard choices for the prior are

$$\mathrm{p}(\theta) = \exp(-||\theta||^2)$$

and

$$\mathrm{p}(\theta) = \exp(-||\theta||_1)$$

They result in L^2 and L^1 Regularization. (Again variances and normalization is omitted)

Decision process: When should I use which regularisation?

- **Goal**: Choice of suitable regularization method based on problem characteristics, available data and prior knowledge.
- No universal method, just a proposal.

Decision

Szenario	Empfehlung	Hinweise
Small data – many features	L2/L1, Early Stopping, simple architecture	Standardization, Cross-Validation for λ
Structured Outputs	TV/Laplace	Output Regularization is often helpful
Strong Prior Knowledge	Regularization by Design	Limit capacity, use inductive bias
Optimization unstable	Gradient-Clipping, Max-Norm, Non-Negativity if necessary	Smaller Learning Rate

Choice of Hyperparameters

- Normalization:
  - Divide the output regularization by the number of output components (e.g. pixel) -> λ is scale-invariant)
  - For sums over samples: use mean instead
- Search:
  - Logarithmic grid instead of linear grid
  - Coarse-to-fine
  - Cost-effective and robust: few epochs for a broad search
- Stability:
  - Stable Optimizers (later)
  - Early Stopping

Hyperparameter Guidelines

- L^2: $1e-6 \ldots 1e-2$
- L^1: $1e-6 \ldots 1e-3$
- Max-Norm: 1 … 3 (per layer)
- Dropout-rate: 0.1 … 0.5
- Patience for early stopping: 5-10
- Learning rate schedule helps

Hyperparameter Choice

- Split: Train/Val/Test clean separation; stratification for classification
- Coarse sweep per regularizer (λ on log-grid)
- Refine combinations only in top 2–3 areas
- Average over 3–5 seeds if N is small
- Retrain final model on Train+Val; Only report once on Test-set

Diagnosis of Under vs. Over-Regularization

- Under-Regularization (λ too small):
  - High variance, big gap between Train and Val, unstable boundaries
  - Action: increase λ ; lower capacity; more aggressive early stopping
- Over-Regularization (λ too high):
  - High bias, both are poor (Train and Val), blurred/too smooth outputs
  - Action: reduce λ ; increase capacity; data augmentation could help
- Output-Regularization specifically:
  - Too much TV: Oversmoothed structures; no fine details
  - Too little TV: high frequency components / noise

Practical Shortcuts

- Standard-Baseline:
  - Start with small L^2 (1e−4), early stopping, no Dropout
  - Then: add output-regularization if the outputs are structured
- Check Scaling:
  - Input-standardization; check loss-scales (data-term vs. regularization term)
  - Take the mean of the regularization term per batch (not sum)
- Documentation:
  - Log all λ and seeds; save training curves; use reproducible splits

Cross Validation

FH KUFSTEIN TIROL
University of Applied Sciences

Cross-Validation

- Unter Cross-Validation versteht man Techniken um herauszufinden wie ein machine learning Modell auf ungesehen Daten performt

- Cross-Validation kann verwendet werden um
  - Das Modell zu evaluieren
  - Das Modell aus verschiedenen Modellen auszuwählen
  - Für Parameter Tuning
  - Um overfitting zu vermeiden

Cross-Validation

- Wie funktionierts:
  - Der Datensatz wird in mehrere Teile geteilt
  - Das Modell wird auf einigen trainiert und auf den anderen getestet
  - Das wird einige Male wiederholt indem verschiedene Teile ausgewählt werden
  - Die finale Perfomance wird als Durchschnitt der einzelnen Validierungsschritte ermittelt

Varianten

- **Hold-Out Cross-Validation:** Der Datensatz wird in zwei Teile geteilt, typischerweise 70-80% um das Modell zu fitten und 20-30% um es zu testen

- **K-Fold Cross Validation:** Datensatz wird in K gleiche Teile geteilt. Das Modell auf $K - 1$ Folds trainiert und auf dem Übrigen getestet. Dieser Prozess wird K mal wiederholt, jedes mal mit unterschiedlichem Test set. Die finale Performance ist der Durchschnitt der K Durchgänge.

- **Stratified Cross Validation:** Ist eine Variation der K-Fold Cross Validation bei der, der Prozentsatz der Klassen in den Splits gleich ist.

- **Leave One Out Cross Validation:** Jeder Datenpunkt wird einmal als Test set verwendet und auf den übrigen trainiert. Das Modell wird N mal trainiert.

Bootstrapping

Resampling to quantify uncertainty without closed-form sampling distributions

FH KUFSTEIN TIROL

University of Applied Sciences

Bootstrapping: core idea

- Many estimators have complicated sampling distributions.
- Bootstrap approximation: treat the observed dataset as an empirical population.
- Repeatedly sample n observations with replacement, fit the model, and recompute the statistic.
- The variability over bootstrap replicates approximates estimator uncertainty.
- Useful for standard errors, confidence intervals, model stability, and performance metrics.

Pseudocode: nonparametric bootstrap

```
Input: data D = {z₁, …, zₙ}, statistic S, replicates B

for b = 1, …, B:
    draw Dᵇ by sampling n observations from D with replacement
    compute sᵇ = S(Dᵇ)
end

return distribution {s¹, …, sᴮ}

Common outputs:
    standard error = std({sᵇ})
    percentile CI = quantiles of {sᵇ}
```

Use the same full learning pipeline inside S: preprocessing, feature selection, tuning if required.

Bootstrap confidence intervals

- Normal approximation: statistic ± 1.96 × bootstrap standard error (standard deviation of bootstrap statistic). Assumption: normal distribution
- Percentile interval: directly use lower and upper bootstrap quantiles.
- BCa interval: bias-corrected and accelerated; more accurate but more complex. If the bootstrap distribution is biased or skewed.
- For performance estimates, resampling should respect data structure and leakage risks.

Bootstrapping in statistical learning

- Coefficient uncertainty: how stable are estimated parameters?
- Feature importance: how often does a variable remain important across samples?
- Prediction uncertainty: distribution of predictions over bootstrap models.
- Model evaluation: bootstrap distribution of AUC, accuracy, RMSE, MAE, etc.
- Bagging: average bootstrap-trained models to reduce variance.

Pseudocode: bootstrap model evaluation

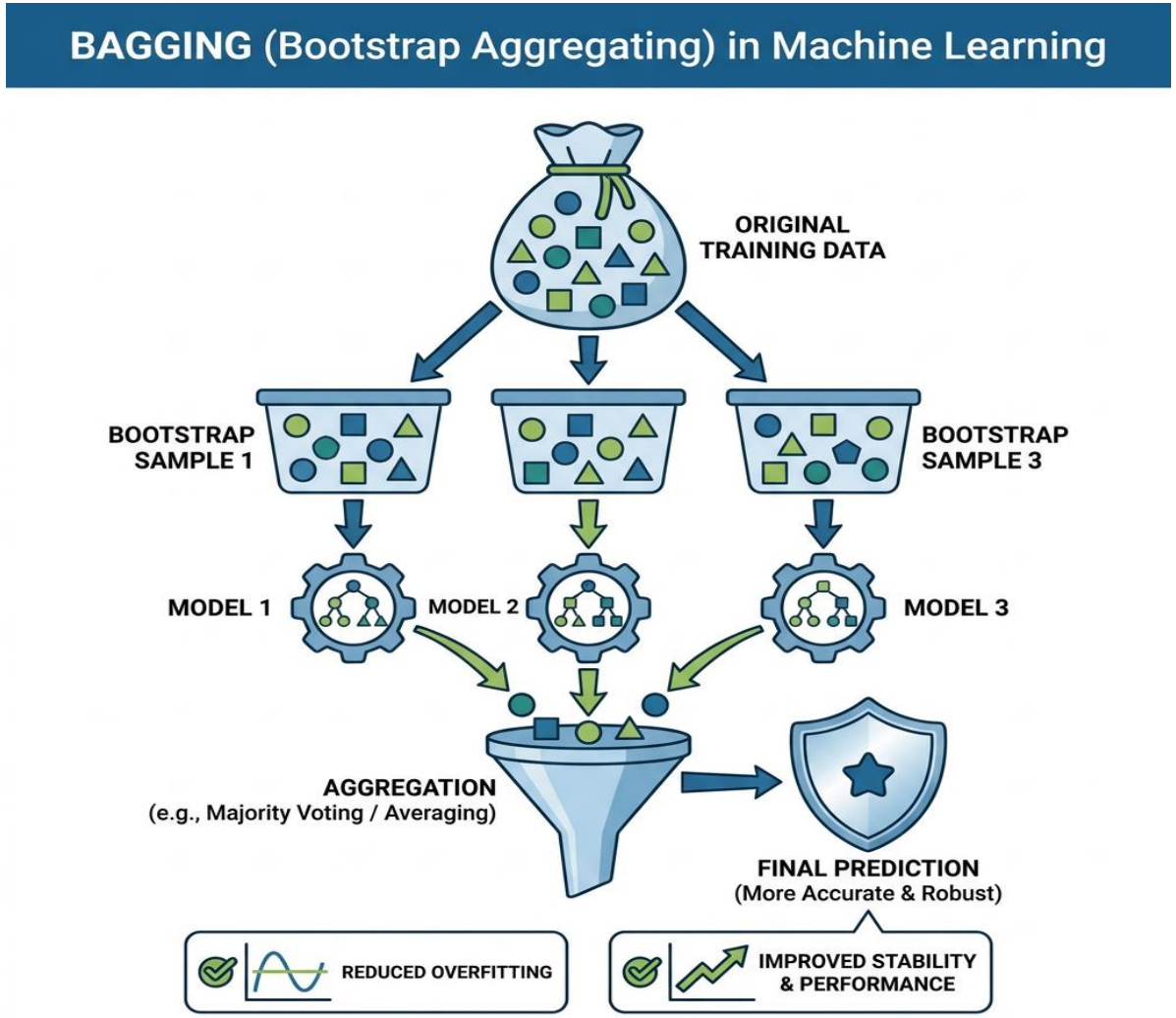
```
Input: data D, training algorithm A, metric M, replicates B

for b = 1, …, B:
    sample bootstrap training set D_train^b from D
    define out-of-bag set D_oob^b = observations not sampled
    fit model θ^b = A(D_train^b)
    evaluate m^b = M(θ^b, D_oob^b)
End

return mean({m^b}), CI({m^b})
```

Out-of-bag evaluation is convenient, but cross-validation is often preferred for model selection.

Bagging


BAGGING (Bootstrap Aggregating) in Machine Learning
ORIGINAL TRAINING DATA
BOOTSTRAP SAMPLE 1
BOOTSTRAP SAMPLE 3
MODEL 1
MODEL 2
MODEL 3
AGGREGATION
(e.g., Majority Voting / Averaging)
FINAL PREDICTION
(More Accurate & Robust)
REDUCED OVERFITTING
IMPROVED STABILITY & PERFORMANCE

Optimization

How statistical learning models are fitted

FH KUFSTEIN TIROL
University of Applied Sciences

Learning as optimization

- Training usually means minimizing an empirical objective.
- Typical objective: loss on data + regularization term.
- Decision variables: parameters θ, sometimes also latent variables or hyperparameters.
- Algorithm choice depends on smoothness, dimensionality, stochasticity, constraints, and curvature.

Unconstrained minimization

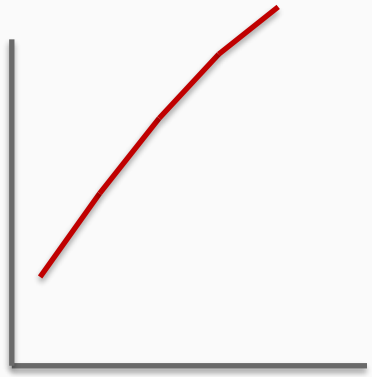
Basic mathematical problem:

$$\textbf{minimize } f(\theta) \textbf{ over } \theta \in \mathbb{R}^d$$

- Global minimum: $f(\theta^*) \leq f(\theta)$ for all feasible θ.
- Local minimum: $f(\theta^*) \leq f(\theta)$ in a neighborhood around θ^*.
- Strict minimum: inequality is strict for all other nearby points.
- Flat minimum: many nearby points share almost the same objective value.

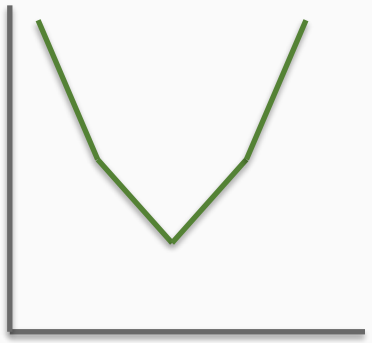
Optimization landscapes: typical cases

No minimum



infimum exists,
but is not attained

Single minimum

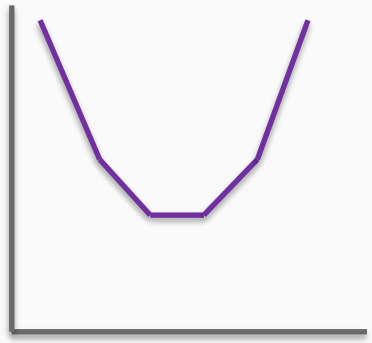


one best point

Multiple minima

several equally good
points

Flat areas



many nearby points
with same loss

Possible minimization cases

Case	Description	Example intuition
No minimum	Infimum exists but is not attained, or f is unbounded below.	f(x)=exp(x) has infimum 0 but no x reaches it.
Single minimum	Exactly one optimizer.	Strictly convex quadratic bowl.
Multiple minima	Several separated optimizers.	Symmetries in neural networks.
Flat area	Continuum of minimizers or near-minimizers.	Redundant parameters or overparameterized models.

Convert maximization to minimization

Most optimization software is written for minimization:

$$\textbf{maximize } \mathbf{g}(\boldsymbol{\theta}) \Leftrightarrow \textbf{minimize } \mathbf{f}(\boldsymbol{\theta}) = -\mathbf{g}(\boldsymbol{\theta})$$

- Maximum likelihood: maximize log-likelihood $\ell(\theta)$.
- Equivalent loss: minimize negative log-likelihood $-\ell(\theta)$.

- The optimizer is identical; only the sign of the objective and gradient changes.
- If $f = -g$, then $\nabla f = -\nabla g$.

Convexity

A function is convex if every line segment lies above the graph:

$$f(\lambda\alpha + (1-\lambda)\beta) \leq \lambda f(\alpha) + (1-\lambda)f(\beta), \qquad \text{for } 0 \leq \lambda \leq 1$$

Geometric meaning: no hidden valleys below the chord between two points

Examples: squared loss in linear regression, logistic regression objective with convex regularization

Non-examples: most deep neural network training objectives

What convexity means for minimization

Good news

Every local minimum is also global

Stationary point $\nabla f(\theta)=0$ is sufficient for global optimality in differentiable convex problems

Strong convexity gives a unique minimizer and often faster convergence guarantees

Important limits

Convex does not mean easy if dimension or data size is huge

Non-smooth convex losses need subgradients or proximal methods

Flat directions may produce non-unique minimizers
Nonconvex methods can still work very well empirically

Deep learning objectives are usually non-convex!

First-order methods

Using gradients without explicitly forming curvature matrices

FH KUFSTEIN TIROL

University of Applied Sciences

Gradient descent

Move opposite to the gradient of the objective:

$$\boldsymbol{\theta_{t+1} = \theta_t - \eta_t \nabla f(\theta_t)}$$

- η_t is the learning rate or step size.
- Large η: fast but may overshoot or diverge.
- Small η: stable but slow.
- Convergence depends strongly on curvature and conditioning.

Step-size strategies

Fixed or scheduled

Constant learning rate for simple problems

Decay schedules: step decay, exponential decay, cosine decay

Warm-up can stabilize early iterations in large models

Line search

Choose η by testing candidate steps

Armijo / Wolfe conditions balance decrease and curvature

Often used in deterministic optimization; less common in noisy mini-batch training

Pseudocode: gradient descent

```
Input: differentiable objective f, initial θ₀, step sizes ηₜ

for t = 0, 1, 2, … until stopping criterion:
    gₜ = ∇f(θₜ)
    θₜ₊₁ = θₜ − ηₜ gₜ
end

return θₜ
```

Stopping criteria: small gradient norm, small objective change, validation metric, or iteration budget.

Momentum: accumulating a velocity vector

- Large consistent components are amplified; oscillating components cancel out.
- Plain gradient descent can zig-zag across narrow valleys

Momentum update

$$v_{t+1} = \beta v_t + \nabla f(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta_t v_{t+1}$$

Momentum

Momentum introduces a velocity that averages recent gradients:

$$\mathbf{v_{t+1}} = \boldsymbol{\beta}\mathbf{v_t} + \boldsymbol{\nabla}\mathbf{f}(\boldsymbol{\theta_t}), \qquad \boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \boldsymbol{\eta_t}\mathbf{v_{t+1}}$$

- β controls memory; common values are around 0.9.
- Reduces zig-zagging in directions where gradients alternate.
- Accelerates movement along directions with consistent gradients.
- Very useful for ill-conditioned objectives.

Pseudocode: gradient descent with momentum

```
Input: objective f, initial θ₀, learning rate η, momentum β
v₀ = 0

for t = 0, 1, 2, …:
    gₜ = ∇f(θₜ)
    vₜ₊₁ = β vₜ + gₜ
    θₜ₊₁ = θₜ − η vₜ₊₁
end

return θₜ
```

Some libraries use $v = \beta v - \eta g$ and $\theta = \theta + v$; this is algebraically equivalent.

Nesterov acceleration

Look ahead before computing the gradient:

$$v_{t+1} = \beta v_t + \nabla f(\theta_t - \eta \beta v_t), \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

- Gradient is evaluated at a predicted future position.
- Can correct the direction earlier than classical momentum.
- For smooth convex problems, accelerated methods can improve worst-case rates.
- In deep learning, the practical difference to momentum is problem-dependent.

Pseudocode: Nesterov accelerated gradient

```
Input: objective f, initial θ₀, learning rate η, momentum β
v₀ = 0

for t = 0, 1, 2, …:
    yₜ = θₜ − η β vₜ            # look-ahead point
    gₜ = ∇f(yₜ)
    vₜ₊₁ = β vₜ + gₜ
    θₜ₊₁ = θₜ − η vₜ₊₁
end

return θₜ
```

Stochastic and adaptive first-order methods

Scaling optimization to large datasets and noisy gradients

FH KUFSTEIN TIROL
University of Applied Sciences

Stochastic gradient descent

Use a minibatch gradient instead of the full gradient:

$$\theta_{t+1} = \theta_t - \eta_t \nabla \ell_B(\theta_t)$$

- Each update uses a small minibatch B instead of all observations.
- Noise can help escape shallow local minima and saddle regions.
- Requires learning-rate schedules and careful batching.
- Dominant method for large-scale neural network training.

SGD practical choices

Choice	Effect	Typical guidance
Batch size	Controls gradient noise and hardware efficiency	Start with powers of 2; larger batches often need larger learning rates
Learning-rate decay	Reduces noise near the optimum	Use validation curves; cosine or step decay are common
Shuffling	Avoids systematic update bias	Shuffle each epoch for IID-style training
Early stopping	Acts as regularization	Monitor validation loss, not only training loss

Pseudocode: stochastic gradient descent

```
Input: dataset D, loss ℓ, initial θ₀, learning rates η_t, batch size m

for epoch = 1, 2, …:
    shuffle D
    for each minibatch B of size m:
        g = (1 / |B|) Σ_{z in B} ∇ℓ(θ; z)
        θ = θ − η_t g
    end
end

return θ
```

Full-batch GD is the special case B = D. Online SGD is the special case |B| = 1.

Why adaptive optimizers?

- Different parameters can have gradients on very different scales
- Sparse features may need larger effective learning rates
- Adaptive methods maintain running statistics of past gradients
- They often reduce tuning effort, but the best final generalization is problem-dependent

Adaptive methods: coordinate-wise learning rates

AdaGrad

$$G_t = G_{t-1} + g_t^2$$

$\frac{\eta}{\sqrt{G_t}}$: strong decay for frequent features

AdaDelta

Exponential moving average (EMA) of g^2 and $\Delta\theta^2$

no global learning rate needed in original form

Adam

$$m_t = \mathrm{EMA}(g), v_t = \mathrm{EMA}(g^2)$$

momentum + adaptive scaling + bias correction

AdaGrad in detail: learning rates per coordinate

Update rule

$$g_t = \nabla_\theta \ell_{B_t}(\theta_{t-1})$$
$$G_t = G_{t-1} + g_t \odot g_t$$
$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{G_t + \varepsilon}}$$

Coordinate form:

$$\theta_{t,i} = \theta_{t-1,i} - \eta \frac{g_{t,i}}{\sqrt{G_{t,i} + \varepsilon}}$$

Intuition

- Keeps a running sum of squared gradients for every parameter.
- Large or frequent gradients ⇒ denominator grows ⇒ smaller future steps.
- Rare features can keep comparatively large effective learning rates.

When it works well / limitations

- Strong for sparse, high-dimensional convex problems such as text features.
- No single coordinate has to share the same effective step size.
- Learning rates only decrease; after many iterations updates may become too small.
- η and ε still matter; often needs scheduling or early stopping.

Key message: AdaGrad is a diagonal preconditioner: it rescales each coordinate by its historical gradient magnitude.

Pseudocode: AdaGrad

Input: initial θ_0, learning rate η, small ε
$G_0 = 0$

for $t = 1, 2, \ldots$:
    $g_t = \text{stochastic\_gradient}(\theta_{t-1})$
    $G_t = G_{t-1} + g_t \odot g_t$
    $\theta_t = \theta_{t-1} - \eta \cdot g_t / (\sqrt{G_t + \varepsilon})$
end

return θ_t

Coordinates with frequent large gradients automatically receive smaller effective step sizes.

AdaDelta in detail: windowed adaptation instead of decay to zero

Update rule

$$\mathrm{E}[g^2]_t = \rho \mathrm{E}[g^2]_{t-1} + (1-\rho) g_t^2$$

$$\mathrm{RMS}[g]_t = \sqrt{\mathrm{E}[g^2]_t + \varepsilon}$$

$$\mathrm{RMS}[\Delta\theta]_t = \sqrt{\mathrm{E}[\Delta\theta^2]_t + \varepsilon}$$

$$\Delta\theta_t = -\frac{RMS[\Delta\theta]_{t-1}}{RMS[g]_t} g_t$$

$$\mathrm{E}[\Delta\theta^2]_t = \rho \mathrm{E}[\Delta\theta^2]_{t-1} + (1-\rho)\Delta\theta_t^2$$

$$\theta_t = \theta_{t-1} + \Delta\theta_t$$

What changes compared with AdaGrad?

- Replaces the cumulative sum G_t with exponential moving averages.
- The optimizer "forgets" old gradients, controlled by ρ.
- Original AdaDelta does not require a global learning rate η.

Practical interpretation

- ρ close to 1 gives a long memory; typical value: $\rho \approx 0.95$.
- RMS[Δθ] keeps update units comparable to parameter units.
- More robust than AdaGrad when training is long, but Adam is more common today.
- Can be useful when choosing η is difficult.

Key message: AdaDelta fixes AdaGrad's monotonically shrinking steps by using a moving window of recent gradients and updates.

Pseudocode: AdaDelta

Input: initial θ_0, decay ρ, small ε
$E[g^2]_0 = 0$, $E[\Delta\theta^2]_0 = 0$

for $t = 1, 2, \ldots$:
    $g_t = \text{stochastic\_gradient}(\theta_{t-1})$
    $E[g^2]_t = \rho\, E[g^2]_{t-1} + (1 - \rho)\, g_t^2$
    $\Delta\theta_t = -\sqrt{(E[\Delta\theta^2]_{t-1} + \varepsilon)} \,/\, \sqrt{(E[g^2]_t + \varepsilon)} \cdot g_t$
    $\theta_t = \theta_{t-1} + \Delta\theta_t$
    $E[\Delta\theta^2]_t = \rho\, E[\Delta\theta^2]_{t-1} + (1 - \rho)\, \Delta\theta_t^2$
end

return θ_t

AdaDelta replaces AdaGrad's accumulating sum with exponential moving averages.

Adam in detail: momentum plus adaptive scaling

Update rule

$$g_t = \nabla_\theta \, \ell_{B_t}(\theta_{t-1})$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{first moment}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{second moment}$$

$$\widehat{m_t} = \frac{m_t}{(1-\beta_1^t)}, \widehat{v_t} = \frac{v_t}{(1-\beta_2^t)}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\widehat{m_t}}{\sqrt{\widehat{v_t} + \varepsilon}}$$

Intuition

- m_t behaves like momentum: it smooths noisy gradients and gives direction.
- v_t rescales coordinates with large recent squared gradients.
- Bias correction compensates for $m_0 = v_0 = 0$ in early iterations.

Defaults and caveats

- Common defaults: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.
- Often a strong default for neural networks and noisy minibatches.
- For regularized deep learning, AdamW is usually preferred over L2 inside Adam.
- Learning-rate schedules still matter; validation performance decides.

Key message: Adam combines the direction-stabilizing effect of momentum with coordinate-wise normalization of recent gradients.

Pseudocode: Adam

Input: initial θ_0, α, β_1, β_2, ε
$m_0 = 0$, $v_0 = 0$

for t = 1, 2, …:
    $g_t = \text{stochastic\_gradient}(\theta_{t-1})$
    $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
    $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
    $\hat{m}_t = m_t / (1 - \beta_1^t)$
    $\hat{v}_t = v_t / (1 - \beta_2^t)$
    $\theta_t = \theta_{t-1} - \alpha \hat{m}_t / \sqrt{\hat{v}_t + \varepsilon}$
end

return θ_t

Adam combines momentum-like first moments with adaptive second-moment scaling.

Adaptive optimizer comparison

Method	Main state	Strength	Typical caution
AdaGrad	sum of squared gradients	Sparse and rare features	Learning rate may decay too much
AdaDelta	EMA of gradients and updates	Reduces manual learning-rate tuning	Less common in modern deep learning
Adam	EMA of gradient and squared gradient	Robust default for many neural networks	Can generalize worse than SGD in some settings

Jupyter Notebook

First order methods comparison

FH KUFSTEIN TIROL

University of Applied Sciences

Second-order methods

Using curvature information

FH KUFSTEIN TIROL
University of Applied Sciences

Newton method

Use a quadratic approximation around the current point:

$$\boldsymbol{\theta_{t+1} = -\mathbf{H}(\theta_t)^{-1}\nabla \mathbf{f}(\theta_t)}$$

- H is the Hessian matrix of second derivatives.
- Can converge very quickly near a well-behaved optimum.
- Expensive in high dimensions: storing and inverting H is costly.
- May require damping or line search if H is indefinite or the step is too aggressive.

Second-order intuition

Benefits

Accounts for curvature and parameter scaling

Can take long, confident steps in low-curvature directions

Often fewer iterations than first-order methods

Challenges

Hessian computation and factorization can dominate runtime

Damping or trust regions are needed far from the optimum

Stochastic variants are harder because curvature estimates are noisy

Pseudocode: damped Newton method

```
Input: twice differentiable f, initial θ₀

for t = 0, 1, 2, …:
    g_t = ∇f(θ_t)
    H_t = ∇²f(θ_t)
    solve H_t p_t = −g_t for search direction p_t
    choose step size α_t by line search
    θ_{t+1} = θ_t + α_t p_t
end

return θ_t
```

Damping or trust regions improve robustness far away from the optimum.

Quasi-Newton methods

- Avoid computing the exact Hessian.
- Build an approximation to the inverse Hessian from gradient differences.
- BFGS is a widely used quasi-Newton method for medium-sized problems.
- L-BFGS stores only a limited history of update pairs, making it suitable for larger models.

Pseudocode: BFGS

```
Input: f, initial θ₀, initial inverse Hessian approximation B₀ = I

for t = 0, 1, 2, …:
    gₜ = ∇f(θₜ)
    pₜ = −Bₜ gₜ
    choose αₜ by line search
    θₜ₊₁ = θₜ + αₜ pₜ
    sₜ = θₜ₊₁ − θₜ
    yₜ = ∇f(θₜ₊₁) − gₜ
    update Bₜ₊₁ using sₜ and yₜ
end

return θₜ
```

BFGS update is designed to preserve positive definiteness under standard curvature conditions.

Pseudocode: L-BFGS

```
Input: f, initial θ₀, memory size m
history = empty

for t = 0, 1, 2, …:
    gₜ = ∇f(θₜ)
    pₜ = − two_loop_recursion(gₜ, history)
    choose αₜ by line search
    θₜ₊₁ = θₜ + αₜ pₜ
    sₜ = θₜ₊₁ − θₜ
    yₜ = ∇f(θₜ₊₁) − gₜ
    append (sₜ, yₜ) to history; keep only newest m pairs
end

return θₜ
```

L-BFGS is popular for smooth deterministic objectives with many variables but limited memory.

First-order vs. second-order

First-order methods

Cheap iterations;
scale to massive data and parameters

Natural fit for mini-batches and GPUs

Need learning-rate tuning and often many iterations

Second-order / quasi-Newton

More curvature-aware;
fewer iterations possible

Higher per-iteration cost and more memory

Attractive for smaller smooth problems or final polishing

Other modern optimizers: LION

- LION uses the sign of a momentum-like update instead of scaling by second moments.
- It can be memory-efficient because it keeps fewer auxiliary states than Adam.
- Update directions are coarse but sometimes work well in large neural networks.
- As with all optimizers, performance is empirical and depends on architecture, data, and tuning.

Pseudocode: LION optimizer

```
Input: initial θ₀, learning rate η, β₁, β₂, weight decay λ
m₀ = 0

for t = 1, 2, …:
    gₜ = stochastic_gradient(θₜ₋₁)
    uₜ = β₁ mₜ₋₁ + (1 − β₁) gₜ
    θₜ = θₜ₋₁ − η · sign(uₜ) − ηλθₜ₋₁
    mₜ = β₂ mₜ₋₁ + (1 − β₂) gₜ
end

return θₜ
```

Different implementations vary slightly; the key idea is sign-based updates with momentum.

Initialization

Starting points matter, especially in nonconvex problems

FH KUFSTEIN TIROL

University of Applied Sciences

Why initialization matters

- Convex objectives: initialization affects speed but not the global optimum, if the method converges.
- Nonconvex objectives: initialization can determine which basin of attraction is reached.
- Bad scaling can cause exploding or vanishing activations and gradients.
- Multiple random seeds are important for honest empirical comparisons.

Common initialization strategies

Context	Strategy	Idea
Linear models	zeros or small random values	Convex objective often makes initialization less critical.
Neural networks with tanh	Xavier or Glorot	Keep activation variance roughly stable across layers.
Neural networks with ReLU	He or Kaiming	Compensate for ReLU zeroing roughly half the activations.
Transfer learning	pretrained weights	Start from features learned on related data.
Clustering	k-means++	Spread initial centers to improve stability.

Initialization checklist

- Set and report random seeds when reproducibility matters
- Run multiple seeds for small datasets or unstable models
- Monitor early training: exploding loss often signals too-large learning rate or poor scaling
- Standardize inputs; initialization formulas assume reasonable input scales
- For transfer learning, decide which layers are frozen, reinitialized, or fine-tuned

Constrained optimization

Optimization with feasible sets, equality constraints, or inequality constraints

FH KUFSTEIN TIROL
University of Applied Sciences

Constrained optimization problem

General form:

$$\textbf{minimize } f(\theta) \textbf{ subject to } g_i(\theta) \leq 0 \textbf{ and } h_j(\theta) = 0$$

Feasible set: all θ that satisfy all constraints.

Constraints may encode physics, budgets, fairness, monotonicity, sparsity, or parameter bounds.

The unconstrained minimizer may be infeasible.

The constrained optimum often lies on the boundary of the feasible set.

Lagrangian and KKT intuition

For differentiable constrained problems, optimality balances objective and active constraints:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\lambda}, \mathbf{v}) = \mathbf{f}(\boldsymbol{\theta}) + \sum_{\mathbf{i}} \boldsymbol{\lambda}_{\mathbf{i}} \mathbf{g}_{\mathbf{i}}(\boldsymbol{\theta}) + \sum_{\mathbf{j}} \mathbf{v}_{\mathbf{j}} \mathbf{h}_{\mathbf{j}}(\boldsymbol{\theta}), \qquad \boldsymbol{\lambda}_{\mathbf{i}} \geq \mathbf{0}$$

- Stationarity: gradient of the Lagrangian vanishes at the optimum
- Primal feasibility: θ satisfies the original constraints
- Dual feasibility: inequality multipliers λ_i are non-negative
- Complementary slackness: inactive constraints have $\lambda_i = 0$

Projected and proximal gradient

Projected gradient

$\theta^{+} = \Pi_C(\theta - \eta \nabla f(\theta))$

Projection Π_C is the closest feasible point under a chosen norm

Simple and robust when projection is cheap

Proximal gradient

Handles $f(\theta) + r(\theta)$, where r may be non-smooth

Example: soft-thresholding for L1 regularization

Foundation for many sparse learning algorithms

Projected gradient algorithm

Gradient descent for constrained problems: take a descent step, then project back to the feasible set

When do we need it?

Optimization variable must satisfy constraints $\theta \in C$

A plain gradient step may leave the feasible set

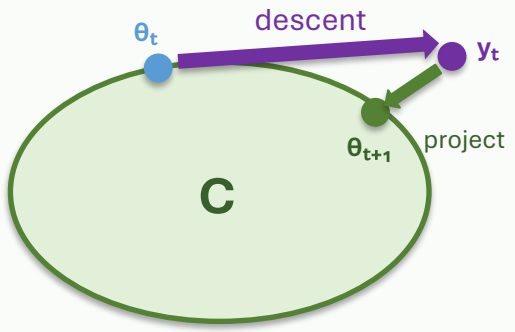
Projection Π_C returns the closest feasible point

θ_t
current feasible point

descent step
$y_t = \theta_t - \eta_t \nabla f(\theta_t)$

projection

Geometric intuition



Core iteration

$$y_t = \theta_t - \eta_t \nabla f(\theta_t) \qquad \theta_{t+1} = \Pi_C(y_t)$$

η_t controls step size; Π_C is often simple for boxes, balls, simplexes, and nonnegativity constraints.
If $C = \mathbb{R}^d$, projection does nothing and the method reduces to ordinary gradient descent.

Pseudocode: projected gradient descent

Input: objective f, constraint set C, projection Π_C, initial $\theta_0 \in C$

for $t = 0, 1, 2, \ldots$:
    $y_t = \theta_t - \eta_t \nabla f(\theta_t)$
    $\theta_{t+1} = \Pi_C(y_t)$
end

return θ_t

Works well when projection onto C is cheap, e.g. boxes, balls, and simplexes.

Proximal operator

Definition

$$\mathbf{prox}_{\eta h}(v) = \mathbf{argmin}_{\theta}\left(h(\theta) + \frac{1}{2\eta}||\theta - v||^2\right)$$

Choose the point θ that balances low penalty h(θ) with staying close to v.

Intuition

- Handles nonsmooth penalties where ordinary gradients are not defined
- The quadratic term prevents jumping too far from v
- Often has a closed-form solution for common regularizers

In proximal gradient

First take a normal gradient step on the smooth part f

$$v_t = \theta_t - \eta \nabla f(\theta_t)$$

Then apply the proximal operator to h

$$\theta_{t+1} = \text{prox}_{\eta h}(v_t)$$

Typical examples

- $h(\theta) = \lambda||\theta||_1$ gives soft-thresholding
- Indicator of a constraint set C gives projection onto C
- $h(\theta) = \lambda||\theta||_2^2$ gives shrinkage toward zero

Key idea: a proximal step is a regularized correction step — optimize the difficult part h while staying near the gradient proposal.

Proximal gradient methods

For composite objectives with a smooth and a nonsmooth part:

$$\textbf{minimize } \mathcal{L}(\boldsymbol{\theta}) = \mathbf{f}(\boldsymbol{\theta}) + \mathbf{h}(\boldsymbol{\theta})$$

- Gradient step handles smooth f.
- Proximal step handles nonsmooth h, e.g. L1 regularization or constraints.
- Update: $\theta_{t+1} = \mathrm{prox}_{\eta h}(\theta_t - \eta \nabla f(\theta_t))$.
- Projected gradient is a special case when h is an indicator of a constraint set.

Pseudocode: proximal gradient descent

Input: smooth f, proximable h, initial θ_0, step size η

for $t = 0, 1, 2, \ldots$:
    $y_t = \theta_t - \eta \nabla f(\theta_t)$
    $\theta_{t+1} = \mathrm{prox}_{\eta h}(y_t)$
end

return θ_t

Example: Lasso uses soft-thresholding as the proximal operator.

Constraint examples in statistical learning

- Non-negativity: coefficients or matrix factors must be ≥ 0
- Simplex constraints: probabilities must be non-negative and sum to 1
- Norm constraints: control model capacity, robustness, or Lipschitz behavior
- Monotonicity constraints: predictions must increase with selected features
- Fairness / resource constraints: enforce application-specific requirements during training

Algorithms for constrained problems

Algorithm family	Core idea	Good fit
Projected gradient	Take a gradient step, then project back to the feasible set	Simple sets: boxes, balls, simplex
Proximal methods	Use a proximal operator for non-smooth penalties or constraints	L1 penalties, sparsity, composite objectives
Penalty methods	Add large cost for constraint violations	Easy to implement; constraints may be approximate
Barrier / interior point	Stay inside feasible region using barrier terms	Smooth convex constrained optimization
Augmented Lagrangian	Combine multipliers with penalties	Equality constraints and difficult feasibility
SQP	Solve sequential quadratic approximations	Nonlinear constrained problems with accurate derivatives
ADMM	Split problem into easier subproblems with consensus updates	Large-scale, distributed, or structured objectives

Algorithm overview

Pseudocode and intuition summary

FH KUFSTEIN TIROL
University of Applied Sciences

Algorithms covered in this chapter

Family	Algorithms	Main idea
First-order	Gradient descent, momentum, Nesterov, proximal gradient	Use gradients
Stochastic/adaptive	SGD, AdaGrad, AdaDelta, Adam, LION	Use minibatches and/or coordinate-wise statistics
Second-order	Newton, BFGS, L-BFGS	Use curvature or curvature approximations
Constrained	Projected gradient, FISTA, penalty, barrier, augmented Lagrangian, ADMM, SQP	Enforce feasibility via projections, penalties, barriers, or subproblems

Optimization takeaways

- State the objective, variables, constraints, and validation protocol clearly.
- Convexity gives strong guarantees; nonconvex problems require diagnostics and multiple runs.
- Gradient descent is simple; momentum and acceleration improve difficult curvature.
- SGD and adaptive optimizers trade deterministic progress for scalable noisy updates.
- Second-order methods can be very efficient when curvature information is affordable.
- Constrained problems need algorithms that respect or penalize feasibility.
- Initialization, learning-rate schedules, and stopping criteria are part of the model design.